

TP : Journalisation (Logging)

Ce TP vise à vous faire pratiquer la configuration et l'utilisation de la journalisation dans une application Laravel.

Nous allons créer un contexte simple : une application de gestion d'utilisateurs basique (comme un système de connexion et d'inscription). Dans ce contexte, nous loggerons différents événements, tels que les tentatives de connexion réussies ou échouées, les erreurs, les informations générales, et des alertes critiques. Nous utiliserons plusieurs niveaux de logs (debug, info, warning, error, critical, etc.) et des canaux différents (fichier par défaut, Slack pour les alertes, et un canal daily pour la rotation des fichiers).

Prérequis :

- Pour le canal Slack : Créez un webhook Slack gratuit (via l'app Incoming Webhooks sur Slack) et ajoutez l'URL dans votre fichier .env.

1 : Installation d'un nouveau projet Laravel 12

Ouvrez un terminal et exécutez les commandes suivantes pour créer un projet Laravel 12 :

```
composer create-project laravel/laravel 013-tp-logging
cd 013-tp-logging
php artisan serve
```

Accédez à **http://127.0.0.1:8000** pour vérifier que l'application fonctionne.

2 : Configuration des canaux de logging

Ouvrez le fichier **config/logging.php**. Par défaut, Laravel utilise le canal stack qui combine plusieurs canaux (comme single pour un fichier unique).

Modifiez le fichier pour ajouter :

- Un canal **daily** pour des logs rotatifs quotidiens.
- Un canal **slack** pour envoyer des alertes critiques à Slack.

Exemple de configuration (ajoutez ou modifiez la section 'channels') :

```
'channels' => [
    'stack' => [
        'driver' => 'stack',
        'channels' => ['single', 'daily', 'slack'],
        'ignore_exceptions' => false,
    ],

    'single' => [
        'driver' => 'single',
        'path' => storage_path('logs/laravel.log'),
        'level' => env('LOG_LEVEL', 'debug'),
    ],

    'daily' => [
        'driver' => 'daily',
        'path' => storage_path('logs/laravel.log'),
        'level' => env('LOG_LEVEL', 'debug'),
        'days' => 14, // Conserve 14 jours de logs
    ],
]
```

```

        'slack' => [
            'driver' => 'slack',
            'url' => env('LOG_SLACK_WEBHOOK_URL'),
            'username' => 'Laravel Logger',
            'emoji' => ':boom:',
            'level' => env('LOG_LEVEL', 'critical'),
        ],
    ],
],

```

Dans votre fichier **.env**, ajoutez :

```

LOG_CHANNEL=stack
LOG_LEVEL=debug
LOG_SLACK_WEBHOOK_URL=https://hooks.slack.com/services/VOTRE/WEBHOOK/URL

```

Cela configure un stack qui envoie les logs vers un fichier unique, un fichier rotatif, et Slack pour les niveaux critiques et supérieurs.

3 : Création du contexte applicatif et de la page d'accueil

Nous allons créer une page d'accueil avec des liens pour tester les logs. Chaque lien pointe vers une route GET qui déclenche un log à un niveau spécifique sur un canal spécifique.

Exécutez :

```
php artisan make:controller LoggingController
```

Ouvrez **app/Http/Controllers/LoggingController.php** et ajoutez le code suivant :

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;

class LoggingController extends Controller
{
    // Méthode pour la page d'accueil
    public function index()
    {
        return view('welcome'); // Nous modifierons welcome.blade.php
    }

    // Méthode pour logger un message via GET
    public function log(Request $request, $channel, $level)
    {
        // Vérification des canaux et niveaux valides (pour sécurité)
        $validChannels = ['stack', 'single', 'daily', 'slack'];
        $validLevels = ['emergency', 'alert', 'critical', 'error', 'warning',
            'notice', 'info', 'debug'];

        if (!in_array($channel, $validChannels) || !in_array($level,
            $validLevels)) {
            return response()->json(['error' => 'Canal ou niveau invalide.'],
                400);
        }

        // Log le message avec contexte (ex. IP de l'utilisateur)
        Log::channel($channel)->{$level}('Message de test au niveau ' . $level .
            ' sur le canal ' . $channel, [
                'ip' => $request->ip(),
                'user_agent' => $request->userAgent(),
            ]
        );
    }
}

```

```

    });

    return response()->json(['message' => 'Log enregistré au niveau ' .
$level . ' sur le canal ' . $channel . '!']);
    }
}

```

Ajoutez les routes dans **routes/web.php** :

```

use App\Http\Controllers\LoggingController;

Route::get('/', [LoggingController::class, 'index']); // Page d'accueil
Route::get('/log/{channel}/{level}', [LoggingController::class, 'log']); // Route
pour logger via GET

```

4 : Création de la vue pour la page d'accueil

Ouvrez ou créez **resources/views/welcome.blade.php** (la vue par défaut de Laravel) et remplacez son contenu par :

```

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>TP Journalisation Laravel</title>
</head>
<body>
    <h1>Bienvenue dans le TP de Journalisation Laravel 12</h1>
    <p>Choisissez un niveau de log et un canal en cliquant sur un lien ci-
dessous. Chaque clic déclenchera un log de test via une requête GET.</p>

    <h2>Canaux disponibles : stack, single, daily, slack</h2>
    <h2>Niveaux disponibles :</h2>
    <ul>
        @foreach (['debug', 'info', 'notice', 'warning', 'error', 'critical',
'alert', 'emergency'] as $level)
            <li>
                <strong>Niveau : {{ $level }}</strong><br>
                @foreach (['stack', 'single', 'daily', 'slack'] as $channel)
                    <a href="/log/{{ $channel }}/{{ $level }}">Logger "{{ $level
}}" sur "{{ $channel }}"</a><br>
                @endforeach
            </li>
        @endforeach
    </ul>

    <p>Après avoir cliqué, vérifiez les fichiers de logs ou Slack pour voir le
message.</p>
</body>
</html>

```

Cette vue génère des liens comme **/log/stack/debug**, **/log/slack/critical**, etc., organisés par niveau pour plus de clarté.

5 : Test des logs avec différents niveaux et canaux

Démarrez le serveur si ce n'est pas déjà fait (php artisan serve).

Accédez à **http://127.0.0.1:8000** pour voir la page d'accueil. Cliquez sur les liens pour tester.

- **Exemples de liens générés :**
 - **/log/stack/info** : Logge un message info sur le canal stack.

- `/log/slack/critical` : Logge un message critical sur Slack (seulement si le niveau minimum est respecté).
- **Niveaux utilisés** : Tous les 8 niveaux standards de Laravel (debug à emergency).
- **Canaux utilisés** : stack (combiné), single (fichier unique), daily (rotatif), slack (notifications).

Ajoutez du contexte (comme l'IP et le user-agent) pour enrichir les logs, comme montré dans le contrôleur.

6 : Vérification des logs

- **Fichiers locaux** : Ouvrez **storage/logs/laravel.log** (ou les fichiers datés pour le canal daily, comme `laravel-YYYY-MM-DD.log`).

Vous y verrez les messages avec timestamps, niveaux, et contexte formaté (ex. : `[20xx-xx-xx 10:00:00] local.INFO: Message de test au niveau info sur le canal stack {"ip":"127.0.0.1","user_agent":"Mozilla/5.0 ..."}).`

- **Slack** : Vérifiez votre canal Slack pour les messages critiques et supérieurs (avec l'emoji `:boom:`).
- **Autres tests** : Essayez des niveaux inférieurs au minimum configuré (ex. debug sur slack si `level=critical`) ; ils ne s'afficheront pas.

7 : Extensions et questions de réflexion

- Essayez d'ajouter un canal **null** pour ignorer les logs en environnement de test.
- Modifiez le niveau minimum dans **config/logging.php** (ex. `'level' => 'error'`) et observez ce qui est enregistré.
- Bonus : Intégrez la journalisation dans un middleware pour logger toutes les requêtes automatiquement.